Week 2 (Days 8-14): Linear algebra for ML

Month 1, Foundations. Time budget: 42 h. Companion reading (optional): *Mathematics for Machine Learning*, ch. 2-4 (https://mml-book.github.io/), and the 3Blue1Brown linear algebra series (https://www.3blue1brown.com/topics/linear-algebra).

Week goals

1. Treat a data point as a vector and a dataset as a matrix. Read every model as "matrix in, matrix out".
2. Compute and interpret dot products, norms, cosine similarity and projections.
3. Do matrix multiplication by hand, in pure Python, in Rust and with BLAS, and explain the roughly 1000x gaps.
4. Solve linear systems properly (solve, not invert), understand conditioning, and fit a line by least squares.
5. Explain eigenvectors and the SVD visually, implement PCA from scratch, and match scikit-learn.

Weekly project (Day 14): image compression with a truncated SVD, with an error vs storage table.

```
data matrix X (m x n)          weights w (n)         predictions (m)
 ? x?^T ?                          ?w??                  ? x?.w ?
 ? x?^T ?          @               ?w??        =         ? x?.w ?
 ? ...   ?                          ?w??                  ? ...    ?
 one row per example            one weight per feature  one dot product per row
```

Day 8: Vectors, norms, dot products, cosine similarity

Review today: Day 7 (+1), Day 5 (+3), Day 1 (+7).

Block 1: Concept lesson (1h15)

Intuition. A vector is a list of numbers with a meaning: an arrow in space, or a point in "feature space". A flat is (area, bedrooms, age). A song might be (tempo, energy, acousticness, vocals). Once things are vectors we can ask geometric questions. *How big is it?* (norm). *How similar are two things?* (dot product, cosine). *How far apart are they?* (distance). Recommendation, search, RAG and attention in Month 4 all reduce to "find vectors that point the same way".

Concrete example. These song vectors are invented for teaching: lofi = (0.3, 0.2, 0.6, 0.1), edm = (0.9, 0.95, 0.05, 0.3), unplugged = (0.35, 0.25, 0.9, 0.8), gym = (0.85, 0.9, 0.1, 0.4). Query edm. Verified output: cosine(edm, gym) = **0.995**, cosine(edm, lofi) = **0.547**, cosine(edm, unplugged) = **0.488**. A "more like this" button would recommend gym first. [ok]

Plain-language maths. - **Dot product:** a.b = Sum a_ib_i, a weighted sum. A linear model's prediction *is* a dot product w.x: each feature times its importance, summed. Geometrically, a.b = ||a||||b||cos ?. It is large when the vectors are long *and* aligned. - **Norms (length):** L2 ||v||? = sqrtSumv_i^2 (straight-line length), L1 = Sum|v_i| (taxicab), Linf = max|v_i|. For v = (3, 4) these are 5, 7 and 4 (verified). L2 and L1 come back on Day 31 as ridge and lasso penalties. - **Cosine similarity** = a.b / (||a||||b||). Only the angle counts, not the length. In text search a long document shouldn't win just for being long. - **Projection** of v onto a unit vector u: (v.u)u, "the shadow of v on u". For v = (3, 4) and u = (1, 0) it is (3, 0), and the angle is 53.13 deg (verified).

Block 2: Worked example (45 min)

cos(edm, gym) by hand: - dot = 0.9.0.85 + 0.95.0.9 + 0.05.0.1 + 0.3.0.4 = 0.765 + 0.855 + 0.005 + 0.12 = **1.745** (the lab prints 1.745 [ok]) - ||edm|| = sqrt(0.81 + 0.9025 + 0.0025 + 0.09) = sqrt1.805 = 1.3435 - ||gym|| = sqrt(0.7225 + 0.81 + 0.01 + 0.16) = sqrt1.7025 = 1.3048 - cos = 1.745 / (1.3435 . 1.3048) = 1.745 / 1.7530 = **0.995** [ok]

Note that `dot(edm, edm) = 1.805 = ||edm||^2`.

Block 3: Coding lab (2h): `code/week-02/day08_vectors_dot.py`

Tasks: 1. Write `cos_matrix(V)` that returns all pairwise cosines of the rows of V in one line. *Hint:* normalise the rows, then compute `Vn @ Vn.T`. 2. Add 20 random "songs" and implement top-k recommendation with `np.argsort`. 3. Show that scaling a vector by 10 changes the dot product and the L2 distance but not the cosine. 4. Systems mindset: for 1M songs x 128-D float32, how many MB does the matrix take, and how long would one query take at about 10 GFLOP/s? (512 MB, and 2.56e8 FLOPs ~=

26 ms.)

Block 4: Practice problems (1h)

1. (1, 2, 3).(4, -5, 6)? *Answer:* 4 - 10 + 18 = 12.
2. Unit vector in the direction (6, 8)? *Answer:* (0.6, 0.8).
3. When is a.b = 0? *Answer:* the vectors are orthogonal (perpendicular), so they carry "independent" information.
4. Cosine of (1, 0) and (-1, 0)? *Answer:* -1.
5. L1 vs L2 distance from (0,0) to (1,1)? *Answer:* 2 vs sqrt2.
6. Express a linear model's prediction for 3 features as a dot product. *Answer:* yhat = w.x + b.

Block 5: Review (1h)

- **Day 7 (+1):** what was the baseline MAE table, and why do baselines matter?
- **Day 5 (+3):** what does a fan-shaped residual plot mean?
- **Day 1 (+7):** compute the Day-1 MSE for w = 4.6, b = 4.5 with a NumPy one-liner (0.604).
- **Self-check:**
- Give the geometric meaning of the dot product.
- Why use cosine for text and embeddings?
- What are the L1, L2 and Linf norms of (-2, 2, 1)? (5, 3, 2)
- What is a projection?
- How is a linear model a dot product?

Going deeper (optional, inside Block 4)

Embeddings preview. Hand-made song vectors become *learned* vectors (embeddings) in Month 4. A model learns them so that items used together point the same way. Retrieval then becomes "normalise, then take the top-k by dot product", exactly the code you wrote today. Libraries such as FAISS speed up the search over millions of vectors with approximate nearest-neighbour indexes.

Distance vs similarity. For unit vectors, ||a - b||^2 = 2 - 2cos ?, so ranking by L2 distance and ranking by cosine give the same order. Prove it by expanding (a - b).(a - b). That is why many systems store normalised vectors and use whichever is faster to compute.

Common mistakes / how to debug

- **Dividing by a zero norm:** add a small epsilon, `/(norm + 1e-12)`.
- **Mixing up `*` and `@`:** `a * b` is element-wise, `a @ b` is the dot product.
- **Unscaled features dominate distances:** area in sq ft (~1000) swamps bedrooms (~3). Standardise first (Day 25).

What to do next

Keep in mind that "similar = small angle". In Month 4, attention computes exactly these dot products between every pair of tokens.

Day 9: Matrices and matrix multiplication (with a Rust comparison)

Review today: Day 8 (+1), Day 6 (+3), Day 2 (+7).

Block 1: Concept lesson (1h15)

Intuition. A matrix has two readings, and you need both: 1. **A table of data:** rows are examples, columns are features (X with shape m x n). 2. **A function that transforms vectors:** W maps an n-D input to a k-D output by taking k dot products. A neural-network layer is exactly this: y = Wx + b.

Matrix multiplication C = AB is "every row of A dotted with every column of B": C_i? = Sum? A_i? B??. The shapes must chain: (m x k)(k x n) -> (m x n). Predicting for a whole dataset at once is one matmul, X (m x n) @ w (n) -> m predictions.

Concrete example. Two flats [[1000, 2], [1500, 3]] (area, bedrooms) and two models stored as columns, W = [[0.05, 0.04], [4, 6]]. X @ W gives a 2 x 2 table where row i holds both models' predictions for flat i. Row 0 is (0.05.1000 + 4.2, 0.04.1000 + 6.2) = (58, 52).

Plain-language properties. - Not commutative: AB != BA in general. The lab checks a random 3 x 3 case: `False`. - Associative: (AB)C = A(BC). The cost differs hugely by order, and this is why frameworks optimise the order of matmul chains. - Transpose: (AB)^T = B^TA^T. You will use this in every gradient derivation (Day 16). - Identity I: AI = A. Inverse A?^1 when it exists: A?^1A = I. - Cost: an (m x k)(k x n) product takes 2mkn FLOPs.

Systems view: why BLAS is thousands of times faster than Python. Verified on this box: - pure-Python triple loop (i-p-j order) at n = 128: **58-74 ms ~= 53-72 MFLOP/s** across my runs - naive Rust, same algorithm (`code/week-02/rust_matmul`, `cargo run --release`): n = 512 in **30.4 ms ~= 8.8 GFLOP/s** - NumPy/BLAS at n = 1024: **6.5-9.3 ms ~= 230-330 GFLOP/s** (timings vary run to run on a shared box)

So compiling the same algorithm gives roughly 120-160x over Python. Blocking for cache, SIMD and multithreading (what BLAS does) gives another ~25-35x over naive Rust. Your C++/Rust instincts (loop order, contiguous memory, cache tiles) are exactly what makes BLAS fast. In Month 6 you will write a tiled kernel in Triton.

Block 2: Worked example (45 min)

Compute by hand: A = [[1, 2], [3, 4]], B = [[0, 1], [1, 0]]. - AB = [[1.0+2.1, 1.1+2.0], [3.0+4.1, 3.1+4.0]] = [[2, 1], [4, 3]], which swaps A's columns. - BA = [[3, 4], [1, 2]], which swaps A's rows. They differ: B acts on whichever side it sits.

Chain cost: A is 1000 x 10, B is 10 x 1000 and v is 1000 x 1. (AB)v costs 2.1000.10.1000 + 2.1000.1000 ~= 22M FLOPs. A(Bv) costs 2.10.1000 + 2.1000.10 = 40K FLOPs. That is a 500x difference from parentheses alone.

Block 3: Coding lab (2h): `code/week-02/day09_matmul.py` and `code/week-02/rust_matmul/`

```
python code/week-02/day09_matmul.py
cd code/week-02/rust_matmul && cargo run --release
```

Tasks: 1. Change the Python loop order to i-j-p (the inner loop then walks down a column of B) and measure. In Python the effect is small because interpreter overhead dominates. Do the same change in Rust, where it matters a lot. Explain why. 2. In Rust, add a blocked (tiled) version with 64 x 64 tiles. Report your GFLOP/s. 3. Verify (AB)^T = B^TA^T numerically with `np.allclose`. 4. Time `(A @ B) @ v` vs `A @ (B @ v)` with the shapes from the worked example.

Block 4: Practice problems (1h)

1. Shape of (3 x 4)(4 x 2)? *Answer:* 3 x 2.
2. FLOPs for (64 x 128)(128 x 256)? *Answer:* 2.64.128.256 = 4,194,304.
3. Is (AB)^T = A^TB^T? *Answer:* no, it is B^TA^T.
4. What does X @ w compute for X (m x n)? *Answer:* m predictions, one dot product per row.
5. Which matrix rotates 2-D vectors by 90 deg? *Answer:* [[0, -1], [1, 0]].
6. Why do frameworks prefer large batched matmuls over many small ones? *Answer:* they amortise overhead and use the cache and SIMD well.

Block 5: Review (1h)

- **Day 8 (+1):** compute the cosine of (1, 1) and (1, 0) (0.707).
- **Day 6 (+3):** what is a group split and when do you need one?
- **Day 2 (+7):** what are the strides of a (3, 4) int64 array? (32, 8)
- **Self-check:**
- Give the two readings of a matrix.
- Shape rule for multiplication?
- FLOP count of a matmul?
- Why is BLAS much faster than a naive compiled loop?
- Give an example where the order of a matmul chain matters.

Common mistakes / how to debug

- **`np.dot` vs `@` vs `*` on 2-D arrays:** prefer `@` for clarity.
- **1-D arrays have no orientation:** `w @ X` and `X @ w` behave differently. Check `.shape`.
- **Benchmarking in debug mode:** Rust must use `--release`, otherwise the timings are meaningless.

What to do next

Read a PyTorch `nn.Linear` docstring (https://pytorch.org/docs/stable/index.html) and notice that it is just y = xW^T + b. You already understand the core of a layer.

Day 10: Linear systems, conditioning, least squares

Review today: Day 9 (+1), Day 7 (+3), Day 3 (+7).

Block 1: Concept lesson (1h15)

Intuition. "Find x such that Ax = b" means: what weights combine A's columns to produce b? With as many independent equations as unknowns there is exactly one answer. In ML we nearly always have **more equations than unknowns**. Seven flats and two parameters (w, b) give seven equations that can't all hold, because the data has noise. So we ask for the x that makes Ax **as close as possible** to b. That is **least squares**, and it is the same as minimising MSE from Day 1.

Concrete example (verified). Put the Day-1 flats into a design matrix X_d = [size, 1] (7 x 2). Least squares gives **w = 4.6814, b = 3.3363**. The grid search on Day 1 found 4.7 and 3.1 (limited by its 0.1 grid). The exact optimum has MSE ~= 0.435, and Day 17 confirms that value with autograd.

Plain-language maths. - **Normal equation:** at the best x, the residual b - Ax is perpendicular to every column of A. Otherwise you could move along a column and do better. So A^T(b - Ax) = 0, which gives **A^TA x = A^Tb**. The lab solves this and gets the same 4.6814 and 3.3363. - **Don't invert.** `np.linalg.solve` (an LU factorisation) or `lstsq` (QR/SVD) are faster and more accurate than `inv(A) @ b`. - **Conditioning.** The condition number ?(A) measures how much input errors get amplified. A rule of thumb: you lose about log?? ? digits of accuracy. Verified: a 12 x 12 Hilbert matrix has ? ~= **1.6e16**, the limit of float64. `solve` gets an error of 0.28 in a true solution of all-ones, and `inv(A) @ b` gets **5.7**. Nearly collinear features (for example area in sq ft *and* area in m^2) cause the same ill-conditioning. This is one reason ridge regression exists (Day 31).

Block 2: Worked example (45 min)

Solve [[2, 1], [1, 3]] x = [5, 10] by elimination. Row 2 minus ½ row 1 gives 2.5 x? = 7.5, so x? = 3. Then 2x? + 3 = 5, so x? = 1. The lab prints [1, 3] [ok].

Least squares by hand on (1,1), (2,2), (3,2) with the model y = wx + b: A^TA = [[Sumx^2, Sumx], [Sumx, m]] = [[14, 6], [6, 3]], and A^Ty = [Sumxy, Sumy] = [1 + 4 + 6, 5] = [11, 5]. Solve: the determinant is 14.3 - 36 = 6. w = (3.11 - 6.5)/6 = 3/6 = 0.5, and b = (14.5 - 6.11)/6 = 4/6 = 0.667. Check with `np.polyfit([1,2,3],[1,2,2],1)`, which gives [0.5, 0.667].

Block 3: Coding lab (2h): `code/week-02/day10_linear_systems.py`

Tasks: 1. Add a duplicated feature (size and 2.size) to the design matrix and try `np.linalg.solve(A^TA, A^Ty)`. It is singular. What does `lstsq` return instead? (The minimum-norm solution.) 2. Add tiny noise (1e-8) to the duplicate column and print ?(A^TA). Explain why the result is now unstable. 3. Use the Week-1 housing CSV: build X = [area, bedrooms, age, city one-hot, 1] and solve least squares. How close are the coefficients to the generator (0.05, 4, -0.4, city premiums)? Watch the one-hot collinearity with the intercept (drop one city). 4. Time `solve` vs `inv @ b` for n = 2000.

Block 4: Practice problems (1h)

1. When does Ax = b have no exact solution? *Answer:* when b is not in the span of A's columns, which is typical with noisy data.
2. Derive the normal equation in two lines. *Hint:* the residual is orthogonal to the column space.
3. ? = 1e10 and float64 has about 16 digits. How many trustworthy digits remain? *Answer:* about 6.
4. Why can one-hot plus intercept make A^TA singular? *Answer:* the one-hot columns sum to the intercept column.
5. Name two numerically stable least-squares solvers. *Answer:* QR and SVD.

Block 5: Review (1h)

- **Day 9 (+1):** compute AB for A = [[1, 0], [2, 1]] and B = [[3], [4]]. *Answer:* [[3], [10]].
- **Day 7 (+3):** what does the noise floor tell you about how much improvement is possible?
- **Day 3 (+7):** why can broadcasting blow up memory?
- **Self-check:**
- Write the normal equation.
- Why avoid matrix inversion?
- What does the condition number measure?
- Give a feature-engineering cause of ill-conditioning.
- What did least squares give for (w, b) on the Day-1 flats?

Common mistakes / how to debug

- Forgetting the column of ones for the intercept.
- `LinAlgError: Singular matrix` means duplicate or collinear features. Inspect `np.linalg.matrix_rank(X)`.
- Huge, oppositely signed coefficients on correlated features are a conditioning symptom.

What to do next

Least squares is linear regression's closed form. In Week 4 you'll reach the same answer with gradient descent, which, unlike the closed form, scales to neural networks.

Day 11: Eigenvalues and eigenvectors

Review today: Day 10 (+1), Day 8 (+3), Day 4 (+7).

Block 1: Concept lesson (1h15)

Intuition. Apply a matrix to every arrow in the plane. Most arrows get rotated *and* stretched. A few special directions only get stretched (or flipped) and are not rotated. Those are the **eigenvectors**, and the stretch factor is the **eigenvalue**: Av = lambdav. They are the matrix's "natural axes".

Why ML cares: - **PCA:** the eigenvectors of a data covariance matrix are the directions of greatest spread (tomorrow). - **Optimisation:** the eigenvalues of the cost's curvature matrix control how fast gradient descent converges and which learning rate diverges (Day 18, Day 25). - **RNNs:** repeatedly multiplying by a matrix makes vectors explode if |lambda| > 1 and vanish if |lambda| < 1. This is the vanishing/exploding gradient problem of Month 3. The lab shows diag(0.9, 1.1) applied 50 times to (1, 1) gives a norm of **117.39**: the 1.1^5^0 ~= 117 part wins and the 0.9^5^0 ~= 0.005 part vanishes.

Concrete example (verified). A = [[2, 1], [1, 2]] has eigenvalues **1 and 3** with eigenvectors (-0.707, 0.707) and (0.707, 0.707). Along the diagonal (1, 1) the matrix stretches by 3. Along the anti-diagonal (1, -1) it leaves lengths unchanged (lambda = 1).

Plain-language maths. Av = lambdav means (A - lambdaI)v = 0 has a non-zero solution, so det(A - lambdaI) = 0. For a 2 x 2 matrix this is a quadratic. For [[2, 1], [1, 2]]: (2 - lambda)^2 - 1 = 0, so lambda = 1 or 3. Symmetric matrices (like covariances) always have real eigenvalues and orthogonal eigenvectors, so use `np.linalg.eigh` for them.

Power iteration: multiply by A and normalise, over and over. The component along the largest-|lambda| eigenvector grows fastest and takes over. In the lab, 20 iterations from (1, 0) reach (0.7071, 0.7071) with Rayleigh quotient v^TAv = **3.0**. Google's original PageRank is power iteration on a giant matrix.

Block 2: Worked example (45 min)

Eigenvectors of [[2, 1], [1, 2]] for lambda = 3: solve (A - 3I)v = [[-1, 1], [1, -1]]v = 0, so v? = v? and v = (1, 1)/sqrt2. For lambda = 1: [[1, 1], [1, 1]]v = 0, so v = (1, -1)/sqrt2. Check A(1, 1) = (3, 3) = 3.(1, 1) [ok].

Now decompose x = (1, 0) = ½(1, 1) + ½(1, -1). Then A?x = ½.3?(1, 1) + ½.1?(1, -1). After a few steps the 3? term dominates, which is why power iteration converges. The speed depends on the ratio |lambda?/lambda?| = 1/3.

Block 3: Coding lab (2h): `code/week-02/day11_eigen.py`

Verified: the covariance of a stretched and rotated 2-D Gaussian cloud has eigenvalues **0.244 and 9.22**, with top direction (-0.798, -0.603). The generator rotated its long axis by the matrix with first row (0.8, 0.6), so this matches up to sign (eigenvector signs are arbitrary). Tasks: 1. Plot the cloud with both eigenvectors drawn as arrows scaled by sqrtlambda. 2. Implement power iteration for the *smallest* eigenvalue. *Hint:* iterate with A?^1 by solving instead of inverting. 3. Count how many iterations power iteration needs to reach 1e-8 accuracy for A = [[2, 1], [1, 2]] vs [[2, 1.9], [1.9, 2]]. Relate the counts to the eigenvalue ratio. 4. Verify trace = sum of eigenvalues and det = product of eigenvalues.

Block 4: Practice problems (1h)

1. Eigenvalues of diag(4, -2)? *Answer:* 4 and -2, with the axis vectors as eigenvectors.
2. Eigenvalues of [[0, -1], [1, 0]] (a 90 deg rotation)? *Answer:* +/-i. Rotations have no real stretch directions.
3. If Av = 5v, what is A^2v? *Answer:* 25v.
4. Why `eigh` instead of `eig` for covariance matrices? *Answer:* symmetric input gives real, sorted, orthogonal output, and it is faster.
5. A recurrent update multiplies by a matrix with largest |lambda| = 0.5 at each of 30 steps. What happens to the signal? *Answer:* it shrinks by about 0.5^3^0 ~= 1e-9 and vanishes.

Block 5: Review (1h)

- **Day 10 (+1):** state the normal equation and solve a 2 x 2 system by hand.
- **Day 8 (+3):** compute the projection of (2, 2) onto (1, 0).
- **Day 4 (+7):** list the cleaning steps applied to the housing table.
- **Self-check:**
- Give the definition of an eigenvector in words.
- Characteristic equation?
- Why does power iteration converge?
- What do covariance eigenvectors represent?
- How do eigenvalues relate to exploding gradients?

Common mistakes / how to debug

- Eigenvectors come back as **columns** of `vecs`, not rows.
- Signs and order are arbitrary (`eigh` sorts ascending). Never hard-code that "the first one is the biggest".
- `eig` on a nearly symmetric matrix can return tiny imaginary parts. Use `eigh` when the matrix is mathematically symmetric.

What to do next

Picture a matrix as "stretch along its eigenvectors". Tomorrow the SVD generalises this to *any* matrix, including non-square data matrices.

Day 12: SVD and PCA from scratch

Review today: Day 11 (+1), Day 9 (+3), Day 5 (+7).

Block 1: Concept lesson (1h15)

Intuition. Every matrix, even a non-square one, does three things in sequence: **rotate, stretch along axes, rotate**. That is the singular value decomposition, X = U Sum V^T. The columns of V are input directions, Sum holds the stretch amounts (singular values, sorted largest first), and the columns of U are output directions. Keeping only the top k singular values gives the **best possible rank-k approximation** of X in the least-squares sense (the Eckart-Young theorem).

PCA (principal component analysis) uses this on centred data. The rows of V^T are the principal directions, and the variance along direction i is sigma_i^2/(m-1). Projecting onto the top k directions compresses n features to k while keeping as much spread as possible. Uses: visualisation (2-D plots of high-D data), denoising, speeding up models, and understanding redundancy.

Concrete example (verified). scikit-learn's bundled digits dataset has 1797 images of 8 x 8 pixels, so 64 features. Explained variance: top 2 components **28.5%**, top 10 **73.8%**, top 20 **89.4%**, top 40 **98.8%**. Two-thirds of the pixels are nearly redundant. Our

from-scratch projection matches `sklearn.decomposition.PCA` exactly (max |difference| 0.0 after accounting for sign flips).

Plain-language maths. - Centre: X_c = X - mean. Without centring, the first component just points at the mean. - SVD: X_c = U Sum V^T. Scores Z = X_c V_k = U_k Sum_k. - Reconstruction: X? = Z V_k^T. The verified rank-5 reconstruction RMSE is **2.923** and rank-20 is **1.409** (pixel values range over 0-16). - Link to Day 11: the covariance is X_c^TX_c/(m-1) = V (Sum^2/(m-1)) V^T. So PCA directions are covariance eigenvectors, and the SVD is the numerically better way to get them.

Block 2: Worked example (45 min)

Take four 2-D points (2, 2), (-2, -2), (1, -1), (-1, 1). They are already centred. The covariance (÷(m-1) = 3) is [[10/3, 6/3], [6/3, 10/3]]: Sumx^2 = 4 + 4 + 1 + 1 = 10 and Sumxy = 4 + 4 - 1 - 1 = 6. Its eigenvalues are 16/3 and 4/3, with directions (1, 1)/sqrt2 and (1, -1)/sqrt2. PC1 explains 16/(16 + 4) = **80%** of the variance. Projected onto PC1, the scores are 2sqrt2, -2sqrt2, 0 and 0: the last two points sit exactly on PC2. Verify with `np.linalg.svd`. The singular values should be sqrt16 = 4 and sqrt4 = 2.

Block 3: Coding lab (2h): `code/week-02/day12_svd_pca.py`

Tasks: 1. Scatter-plot the 2-D PCA scores of the digits coloured by label. Which digits overlap? 2. Plot the first 8 principal components as 8 x 8 images ("eigendigits"). 3. Plot the cumulative explained variance curve and choose k for 95%. 4. Denoise: add Gaussian noise to the digits, reconstruct with k = 20, and compare the errors against the clean images.

Block 4: Practice problems (1h)

1. Why centre before PCA? *Answer:* otherwise PC1 captures the mean offset, not the spread.
2. X is 1000 x 50. What are the shapes of U, Sum and V^T with `full_matrices=False`? *Answer:* 1000 x 50, (50,), 50 x 50.
3. Singular values are [10, 5, 1]. What fraction of variance do the top 2 explain? *Answer:* (100 + 25)/126 = 99.2%.
4. Is PCA supervised? *Answer:* no. It ignores labels, so the top components may not be the most predictive.
5. Should you standardise features before PCA? *Answer:* usually yes, if their units differ (otherwise sq ft dominates bedrooms).

Block 5: Review (1h)

- **Day 11 (+1):** find the eigenvalues of [[3, 0], [0, 1]] and explain what power iteration finds.
- **Day 9 (+3):** compute the FLOPs of a 100 x 100 matmul (2e6).
- **Day 5 (+7):** what does a clustered residual plot suggest?
- **Self-check:**
- Describe the SVD as three geometric steps.
- How do singular values relate to PCA variance?
- What does Eckart-Young say, in words?
- How many digits components give about 90% of the variance? (20)
- Give two uses of PCA.

Going deeper (optional, inside Block 4)

Computing an SVD in practice. Full SVD of an m x n matrix costs about O(mn.min(m, n)). For huge data you rarely need all the components. *Randomised SVD* (`sklearn.utils.extmath.randomized_svd`, or `PCA(svd_solver="randomized")`) projects onto a random low-dimensional subspace first and gets the top-k much faster. Try it on digits with k = 10 and compare the explained variance with the exact result.

Whitening. Dividing the PCA scores by sqrtvariance gives features with identity covariance. Some algorithms like that, but it also amplifies noisy low-variance directions, so use it deliberately.

Common mistakes / how to debug

- Fitting PCA on train+test together is leakage. Fit on train and transform both.
- Sign flips between libraries are normal. Compare absolute values or reconstructions.
- `full_matrices=True` on tall data allocates a huge U. Use False.

What to do next

Low-rank structure is everywhere: LoRA in Month 4 fine-tunes an LLM by learning a *low-rank* update ?W = BA. Today's intuition is the core of that idea.

Day 13: Projections, orthogonality, Gram-Schmidt/QR

Review today: Day 12 (+1), Day 10 (+3), Day 6 (+7).

Block 1: Concept lesson (1h15)

Intuition. Least squares (Day 10) has a geometric picture. The predictions Ax can only lie in the flat "sheet" spanned by A's columns (the column space). The target y usually sticks out of that sheet. The best prediction yhat is the **shadow** of y on the sheet, its orthogonal projection, and the residual is the perpendicular part. Regression *is* projection.

Concrete example (verified). We use random A (6 x 2) and y (6). The projection matrix P = A(A^TA)?^1A^T is **symmetric**, **idempotent** (P^2 = P: projecting twice changes nothing), and the residual is orthogonal to every column (A^Tr = [0, 0] to 1e-10). All three are printed True or [0, 0] in the lab.

Orthonormal bases make everything easy. If Q's columns are orthonormal (Q^TQ = I), then the projection is just QQ^T y. There is no inverse, and the result is numerically stable. **Gram-Schmidt** builds such a Q from any A: take each column, subtract its shadows on the previous q's, then normalise. The result is the **QR decomposition** A = QR, which is what good least-squares solvers use inside. The lab's Gram-Schmidt Q has Q^TQ = I and spans the same space as `np.linalg.qr`.

Plain-language maths. - Projection onto a unit vector q: (q.y)q. - Onto orthonormal q? ... q_k: Sum(q_i.y)q_i = QQ^Ty. - Gram-Schmidt step: v? = a? - Sum_{i<j}(q_i.a?)q_i, then q? = v?/||v?||. - Least squares via QR: Ax ~= y becomes Rx = Q^Ty, a triangular solve, and ?(R) = ?(A), not ?(A)^2. That is why QR beats the normal equation numerically.

Distances. The lab prints L2 = 3.742, L1 = 6 and Linf = 3 for (1, 2, 3). Different norms give different geometry, and you will choose between them for k-NN (Day 36) and regularisation (Day 31).

Block 2: Worked example (45 min)

Gram-Schmidt on a? = (1, 1, 0) and a? = (1, 0, 1): - q? = (1, 1, 0)/sqrt2. - q?.a? = 1/sqrt2. v? = a? - (1/sqrt2)q? = (1, 0, 1) - (½, ½, 0) = (½, -½, 1). ||v?|| = sqrt(¼ + ¼ + 1) = sqrt1.5, so q? = (½, -½, 1)/sqrt1.5. - Check q?.q? = (½ - ½ + 0)/(sqrt2.sqrt1.5) = 0 [ok].

Project y = (1, 2, 3): q?.y = 3/sqrt2 and q?.y = (½ - 1 + 3)/sqrt1.5 = 2.5/sqrt1.5. yhat = (3/sqrt2)q? + (2.5/sqrt1.5)q? = (1.5, 1.5, 0) + (2.5/1.5)(½, -½, 1) = (1.5 + 0.833, 1.5 - 0.833, 1.667) = **(2.333, 0.667, 1.667)**. Residual = (-1.333, 1.333, 1.333). Check that it is ? a?: -1.333 + 1.333 = 0 [ok].

Block 3: Coding lab (2h): `code/week-02/day13_norms_projections.py`

Tasks: 1. Verify the worked example numerically. 2. Solve the Day-1 flats least squares three ways (normal equation, QR with `np.linalg.solve(R, Q.T@y)`, `lstsq`) and compare them. 3. Classical Gram-Schmidt loses orthogonality on ill-conditioned input. Build A with nearly parallel columns and measure ||Q^TQ - I||. Then implement *modified* Gram-Schmidt and compare. 4. Show that the residual of the Day-1 fit sums to 0. *Hint:* the residual is orthogonal to the ones column.

Block 4: Practice problems (1h)

1. Is P = I - A(A^TA)?^1A^T a projection too? Onto what? *Answer:* yes, onto the residual space (the orthogonal complement).
2. Why does an intercept force the residuals to sum to zero? *Answer:* the residual is ? to the ones column.
3. With orthonormal Q, what is the least-squares solution? *Answer:* x = Q^Ty.
4. Why is QR more stable than the normal equation? *Answer:* ?(A^TA) = ?(A)^2.
5. Project (3, 4) onto the line through (1, 1). *Answer:* (3.5, 3.5).

Block 5: Review (1h)

- **Day 12 (+1):** compute the explained-variance fraction from singular values [3, 1]. *Answer:* 9/10.
- **Day 10 (+3):** why avoid `inv`? What ? did Hilbert(12) have?
- **Day 6 (+7):** describe the leakage experiment and its numbers.
- **Self-check:**
- Regression as geometry, in one sentence.
- Name two properties of a projection matrix.
- What is one step of Gram-Schmidt?

- How does QR solve least squares?
- What are the L1, L2 and Linf norms of (1, 2, 3)?

Going deeper (optional, inside Block 4)

Householder vs Gram-Schmidt. Production QR (LAPACK, behind `np.linalg.qr`) uses Householder reflections, not Gram-Schmidt. Each reflection zeroes a whole column below the diagonal in one numerically stable step, so orthogonality is preserved to machine precision even for ill-conditioned matrices. Gram-Schmidt is the clearest way to *understand* QR. Householder is the right way to *compute* it.

Orthogonality in ML. Orthogonal weight initialisation keeps signal norms stable through deep networks (Month 3). Orthogonal features make regression coefficients independent of each other, which is one reason PCA features are easy to interpret in a linear model.

Common mistakes / how to debug

- Forgetting to normalise in Gram-Schmidt leaves the vectors orthogonal but not unit length.
- Building P explicitly for big m is an m x m matrix, which wastes memory. Use QR.

What to do next

Tomorrow you will put everything together on an image, a matrix you can *see*.

Day 14: Week 2 project: SVD image compression, plus Quiz 2

Review today: Day 13 (+1), Day 11 (+3), Day 7 (+7).

Block 1: Concept lesson (1h15)

Intuition. A grayscale image is a matrix of brightness values. Natural images are highly structured: smooth regions and repeated patterns. So most of the image's "energy" sits in a few singular values. Keep the top k singular triplets (u_i, sigma_i, v_i), store k(h + w + 1) numbers instead of h.w, and reconstruct with Sum_i sigma_iu_iv_i^T. This is not how JPEG works (JPEG uses the DCT on 8 x 8 blocks, which you may know from multimedia work), but it is the same principle: **represent the signal in a basis where it is sparse, and drop the small coefficients.**

Plain-language maths. The relative error of the best rank-k approximation is sqrt(Sum_{i>k} sigma_i^2)/sqrt(Sum_i sigma_i^2), so you can read it straight off the singular value spectrum before reconstructing anything.

Block 2: Worked example (45 min)

Verified on the synthetic 128 x 192 image (sinusoids + disc + step + noise):

k	numbers stored	% of original	relative error
2	642	2.6%	0.3322
8	2568	10.4%	0.0912
32	10272	41.8%	0.0456

From k = 8 to k = 32 the error only halves while the storage quadruples. The remaining error is mostly the added random noise, which has no low-rank structure. Diminishing returns are visible straight from the spectrum.

Block 3: Coding lab: project (2h): `code/week-02/day14_week_project_svd_compress.py`

Deliverables (`notes/week02_report.md`): 1. Plot sigma_i on a log scale. Mark where the "elbow" is. 2. Compute the relative error from the singular values alone and confirm it matches the reconstruction error. 3. Load a real photo (any PNG you own) with `matplotlib.image.imread`, convert it to gray, and produce the same table. 4. Compare with a "block" approach: SVD of each 8 x 8 tile with k = 2. Which is better at equal storage on your photo? 5. Bonus (systems): estimate compression time for a 4K frame using the timings from Day 9, and discuss why codecs use fixed transforms (DCT) instead of a per-image SVD.

Suggested timeline (2h). - 0:00-0:25: run the script, then plot the singular values on a log scale and mark the elbow. Write one sentence on how you chose it, since elbows are judgement calls. - 0:25-0:50: compute the relative error from the singular values, sqrt(Sum_i>? sigma_i^2)/sqrt(Sum_i sigma_i^2), and assert that it matches the error of the reconstructed image to about 1e-10. This is the Eckart-Young result made concrete. - 0:50-1:20: repeat on your own photo. Report the storage for rank k, k(m + n + 1) numbers, against m.n for the original. - 1:20-1:45: the 8 x 8 block comparison at equal storage. Show both images side by side at the same budget. - 1:45-2:00: the systems bonus and a proofread.

Rubric (self-grade out of 20).

criterion	points
Singular-value plot on a log scale with the elbow marked and justified	3
Error from the singular values matches the reconstruction error (assert in code)	5
Own-photo table: k, storage ratio, relative error, with the images shown	4
Fair block-SVD comparison at equal storage, with a visual and a verdict	4
Storage formula derived, not just used	2
Honest discussion of why real codecs use fixed transforms	2

What separates a strong report from a weak one. A weak report shows nice pictures. A strong one shows the error identity holding numerically, compares methods *at equal storage* (otherwise the comparison is meaningless), and names a limitation: a per-image SVD must store the basis vectors, which fixed transforms avoid.

Block 4: Practice: Quiz 2 (1h), answers below

Block 5: Review (1h)

- **Day 13 (+1):** Gram-Schmidt on (1, 0) and (1, 1).
- **Day 11 (+3):** why eigenvalues matter for RNNs.
- **Day 7 (+7):** which baseline won in Week 1, and by how much?
- **Self-check:**
- Storage formula for rank k?
- How do you read the error from the singular values?
- Why doesn't noise compress?
- How does this relate to PCA?
- Name an ML method that uses low rank (LoRA).

Going deeper (optional)

SVD vs DCT, from a codec engineer's view. The SVD basis is optimal *for this one image*, but you must store the basis (U and V) as well, and computing it is costly. The DCT basis is fixed, so it costs nothing to transmit and has fast O(N log N) transforms, and for smooth natural images it comes close to optimal. Modern learned codecs and neural compression revisit this trade-off by *learning* a transform from many images. That's an autoencoder, which you'll build in Month 4.

Colour images. Do the SVD per channel, or convert to YCbCr and spend more rank on luma (Y) than chroma. Human vision is less sensitive to chroma detail, the same trick as 4:2:0 subsampling in video.

Common mistakes / how to debug

- Using RGB without converting to gray, or without doing the SVD per channel.
- Using uint8 images in maths. Convert to float first.

What to do next

Next week is calculus. The question changes from "what is the best fit?" (closed form) to "which way should I move to improve?" (gradients). That question scales to billions of parameters.

Weekly quiz: Week 2 (answers below)

1. Compute (2, -1, 3).(1, 4, 2).
2. Cosine similarity of (1, 0) and (1, 1)?
3. Shape of (5 x 3)(3 x 7)^T? (Careful.)
4. FLOPs to multiply two 256 x 256 matrices?
5. Rank the measured speeds: Python loops, naive Rust and BLAS (orders of magnitude from your run).
6. Write the normal equation, and say why `lstsq` is preferred.
7. What does a condition number of 1e12 imply in float64?
8. Eigenvalues of [[2, 1], [1, 2]]?
9. What fraction of variance did the top 10 PCA components of digits explain?
10. Name three properties of an orthogonal projection matrix P (or of its residual).
11. Storage for a rank-k SVD of an h x w image?
12. Why must PCA be fit only on the training set?

Answers

1. 2 - 4 + 6 = 4.
2. 1/sqrt2 ~= 0.707.
3. Trick question: (5 x 3)(3 x 7) is 5 x 7, and its transpose is 7 x 5.
4. 2.256^3 ~= 33.6M FLOPs.
5. Python ~= 50-70 MFLOP/s < naive Rust ~= 8.8 GFLOP/s < BLAS ~= 230-330 GFLOP/s (on this box; yours will differ).
6. A^TA x = A^Tb. `lstsq` and QR avoid squaring the condition number and handle rank deficiency.
7. You may lose about 12 of the ~16 significant digits.
8. 1 and 3.
9. 73.8%.
10. Symmetric, idempotent (P^2 = P), and the residual (I - P)y is orthogonal to the column space.
11. k(h + w + 1).
12. Fitting on test data leaks its statistics into the model (leakage).